TrOCR

Transformer-based Optical Character Recognition with Pre-trained Models

[2109.10282] TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models

Abstract

Раньше для OCR обычно использовали комбинацию из CNN (чтобы вытащить признаки из изображения) и RNN (чтобы генерировать последовательность символов)

Но авторы TrOCR предложили **end-to-end** модель, основанную на предобученных трансформерах (визуальном и текстовом).

Introduction

Обычно ocr состоит из 2-х модулей (text detection и text recognition), в TrOCR сфокусирован второй модуль, когда текст задетекчен

изображение размера **384 × 384** разбивается на последовательность патчей **16 × 16**.

Model Archtecture

encoder

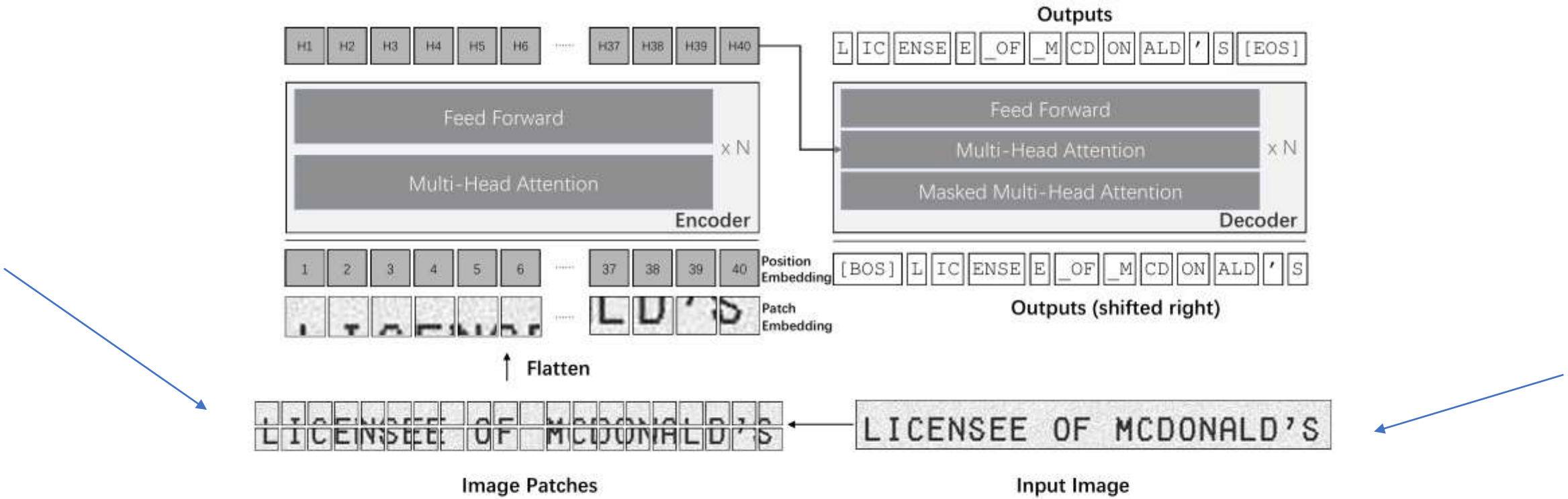


Figure 1: The architecture of TrOCR, where an encoder-decoder model is designed with a pre-trained image Transformer as the encoder and a pre-trained text Transformer as the decoder.

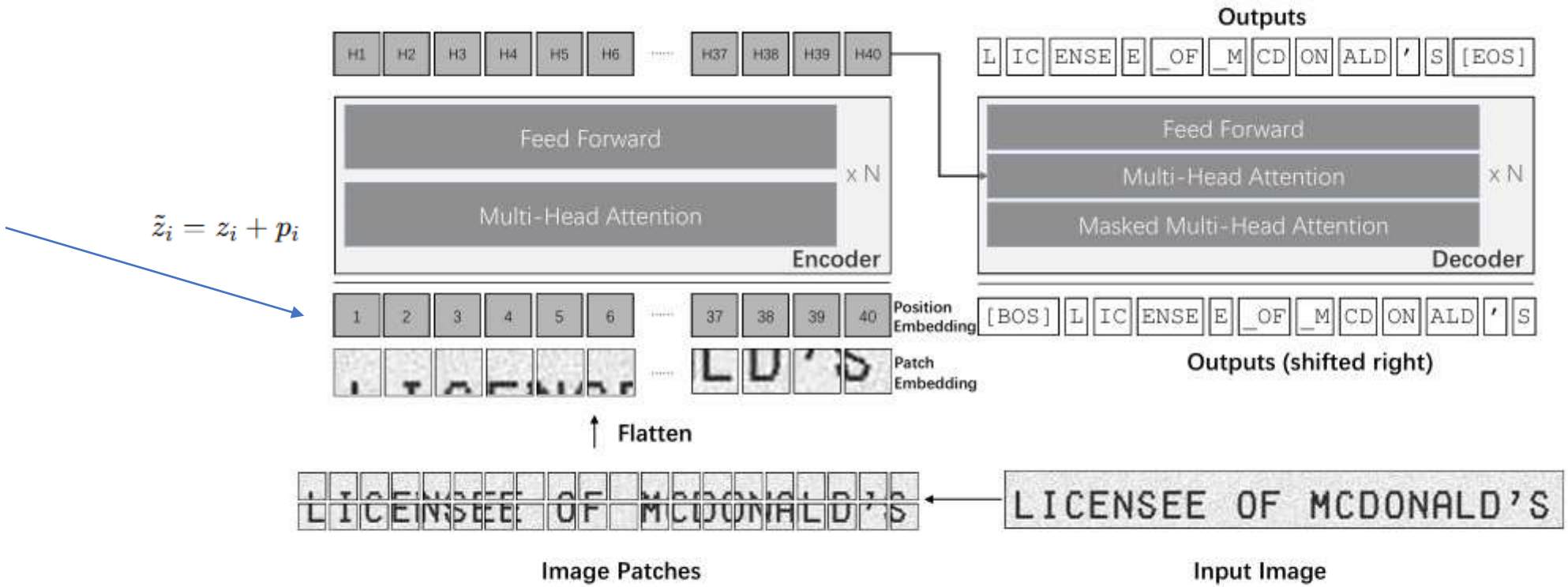


Figure 1: The architecture of TrOCR, where an encoder-decoder model is designed with a pre-trained image Transformer as the encoder and a pre-trained text Transformer as the decoder.

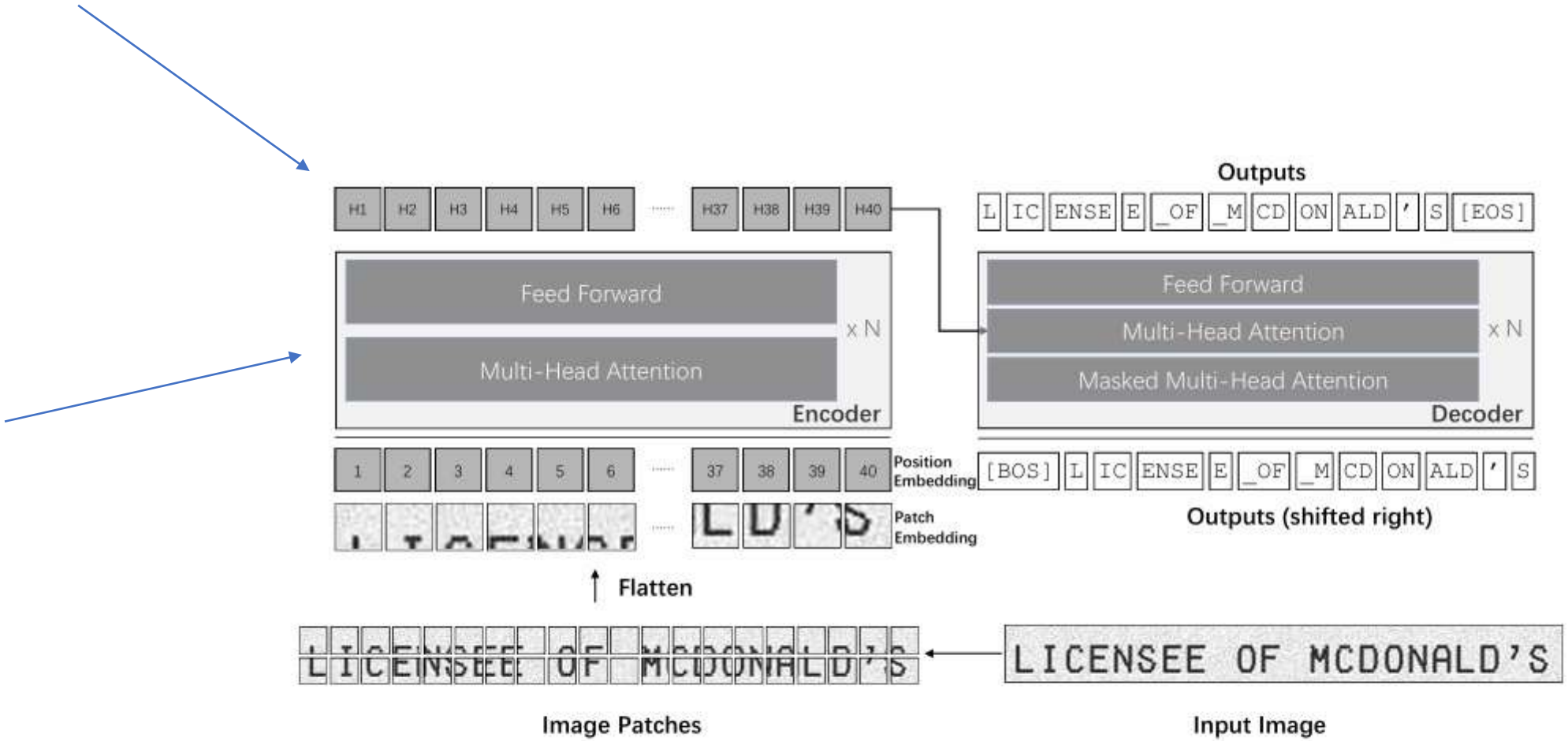


Figure 1: The architecture of TrOCR, where an encoder-decoder model is designed with a pre-trained image Transformer as the encoder and a pre-trained text Transformer as the decoder.

Model Archtecture

decoder

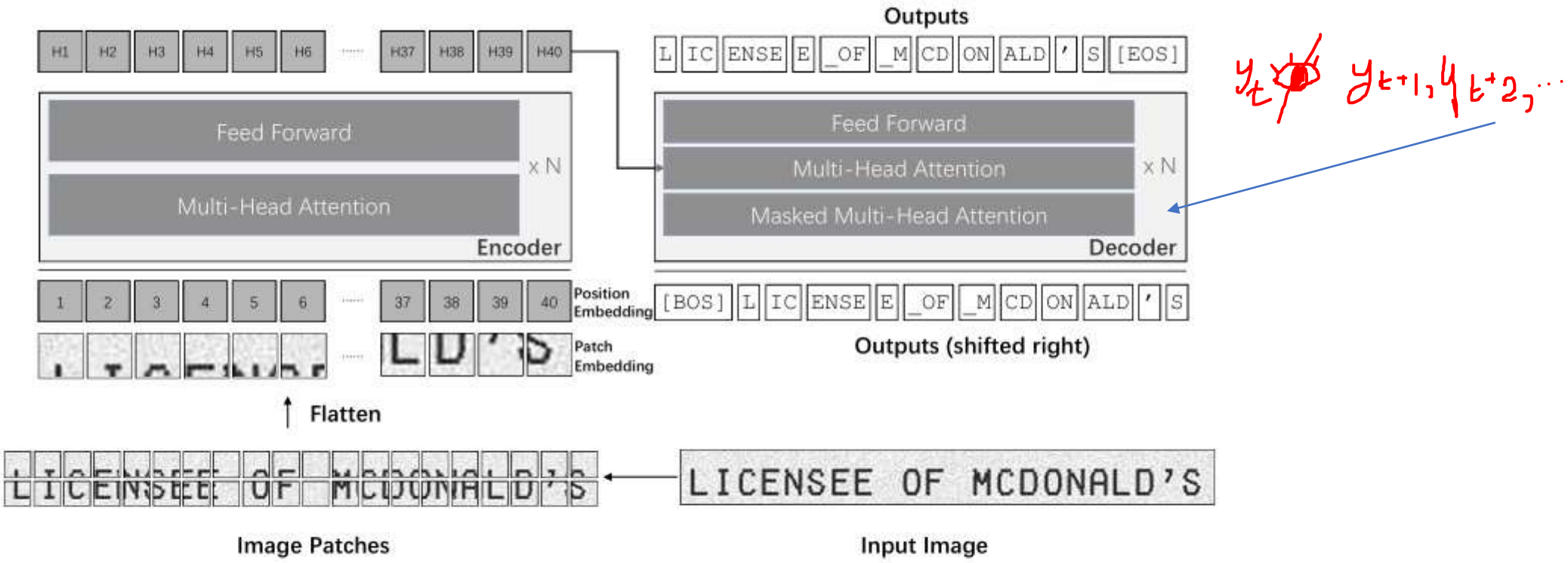


Figure 1: The architecture of TrOCR, where an encoder-decoder model is designed with a pre-trained image Transformer as the encoder and a pre-trained text Transformer as the decoder.

Keys & V

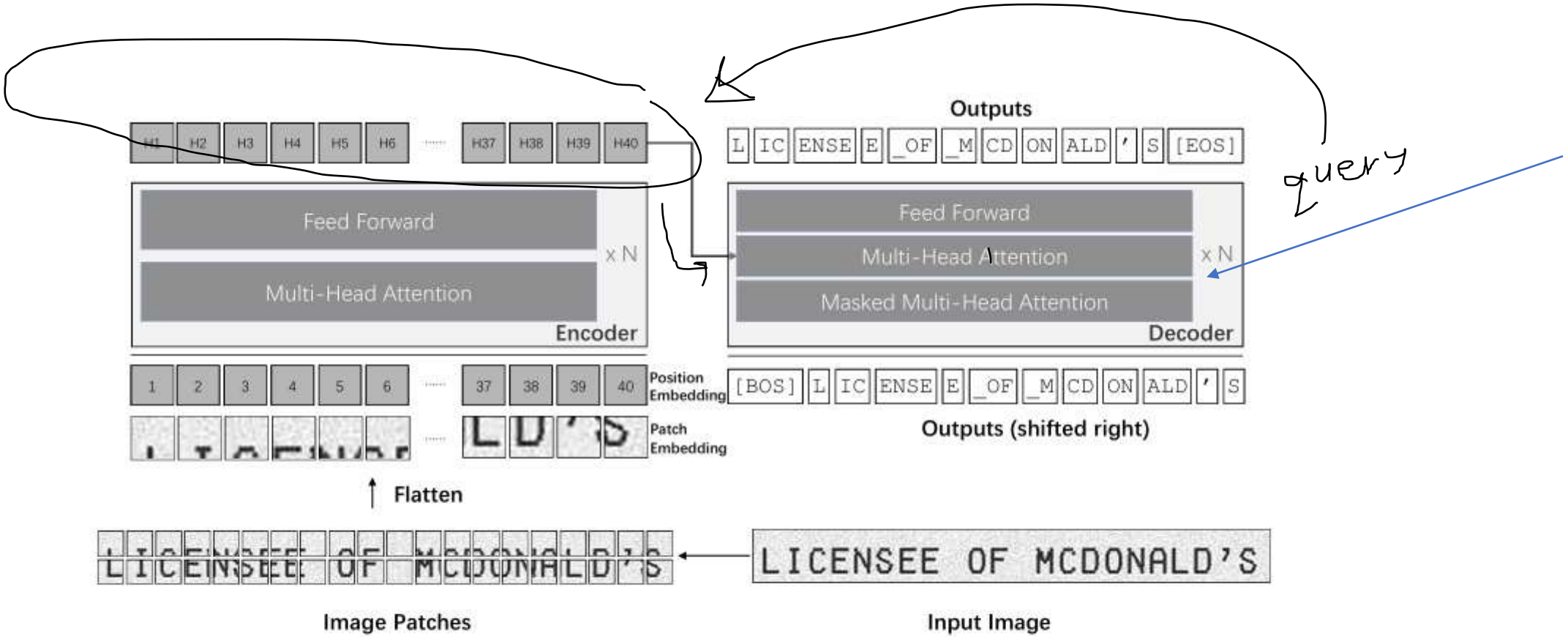


Figure 1: The architecture of TrOCR, where an encoder-decoder model is designed with a pre-trained image Transformer as the encoder and a pre-trained text Transformer as the decoder.

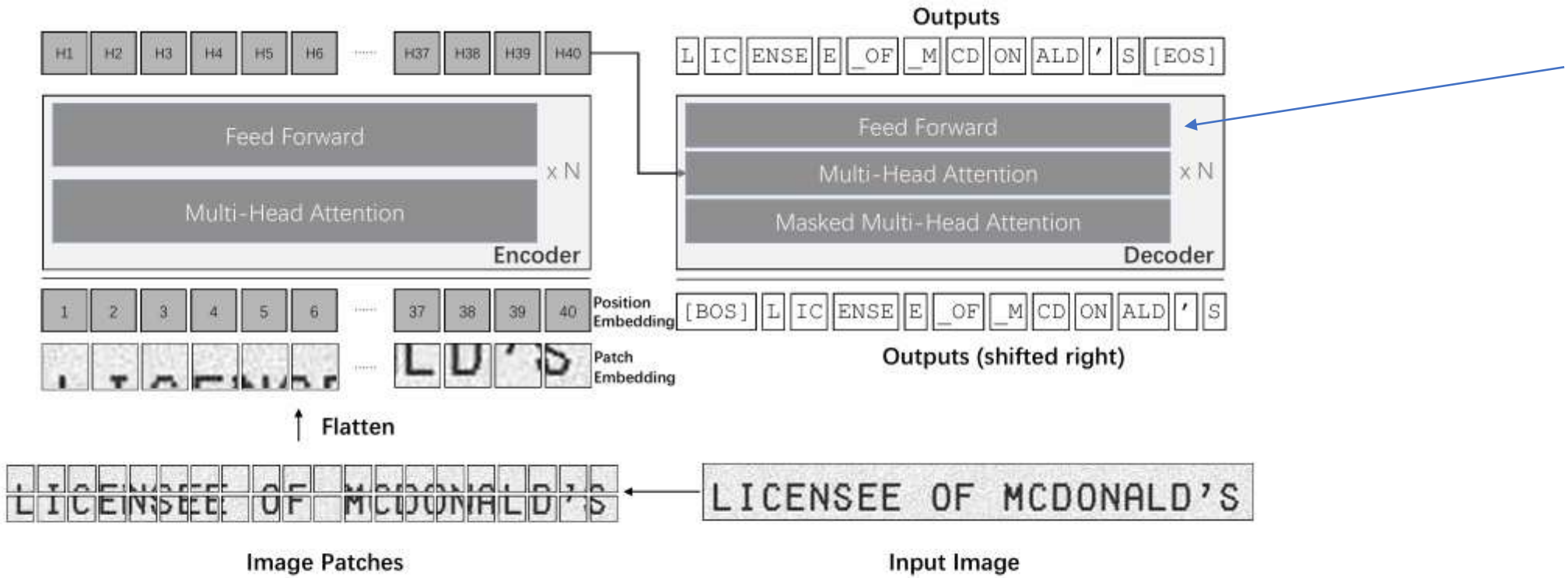


Figure 1: The architecture of TrOCR, where an encoder-decoder model is designed with a pre-trained image Transformer as the encoder and a pre-trained text Transformer as the decoder.

У сверточных нейронок есть проблема: они не могут смотреть глобально, а в трансформерах благодаря механизму attention каждый патч обращает внимание на остальные патчи

скрытое состояние hi

$$h_i = Proj(Emb(Token_i))$$

$$\sigma(h_{ij}) = \frac{e^{h_{ij}}}{\sum_{k=1}^{V} e^{h_{ik}}} \quad for\ j = 1, 2, \ldots, V$$

softmax по словарю размера

The hidden states from the decoder are projected by a linear layer from the model dimension to the dimension of the vocabulary size V, while the probabilities over the vocabulary are calculated on that by the softmax function. We use beam search to get the final output.

Это позволяет находить более вероятную общую последовательность

У енкодера начальные веса берутся из предобученных визуальных моделей DeiT, BEiT

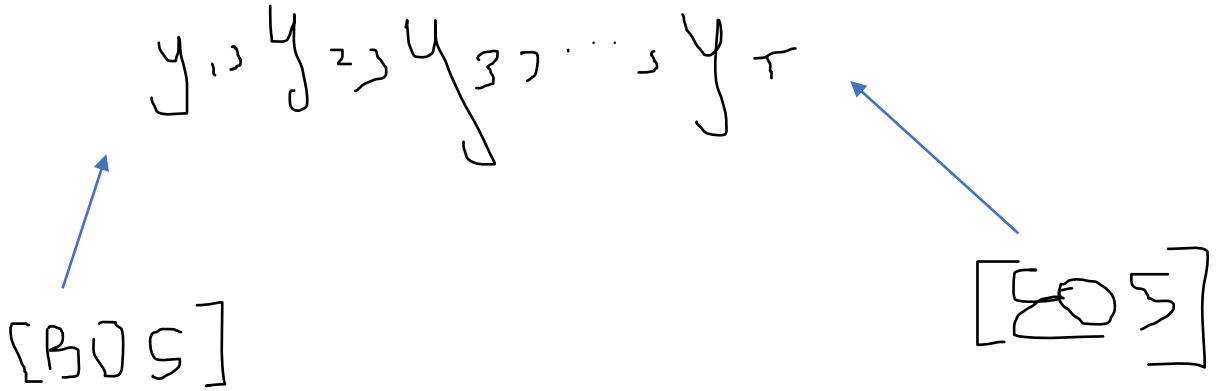
Декодер веса берет из предобученных языковых моделей RoBERTa, MiniLM

авторы берут их веса и аккуратно мапят в структуру decoder-а там, где это возможно. А недостающие параметры инициализируют случайно, так как это encoder-only модели

Model Archtecture

Task pipeline

Есть правильная последовательность:



Максимизировать

$$P(y^* \mid x) = \prod_{t=1}^{T} P(y_t^* \mid y_{<t}^*, x)$$

На основе Y(<t) декодер учится предсказывать сдвиг вправо (yt)

Loss считается обычной кросс-енотропией (отриц сумма логарифма усл вероятности), птмчт на каждом шаге t получаем распределение вероятностей по словарю через софтмакс